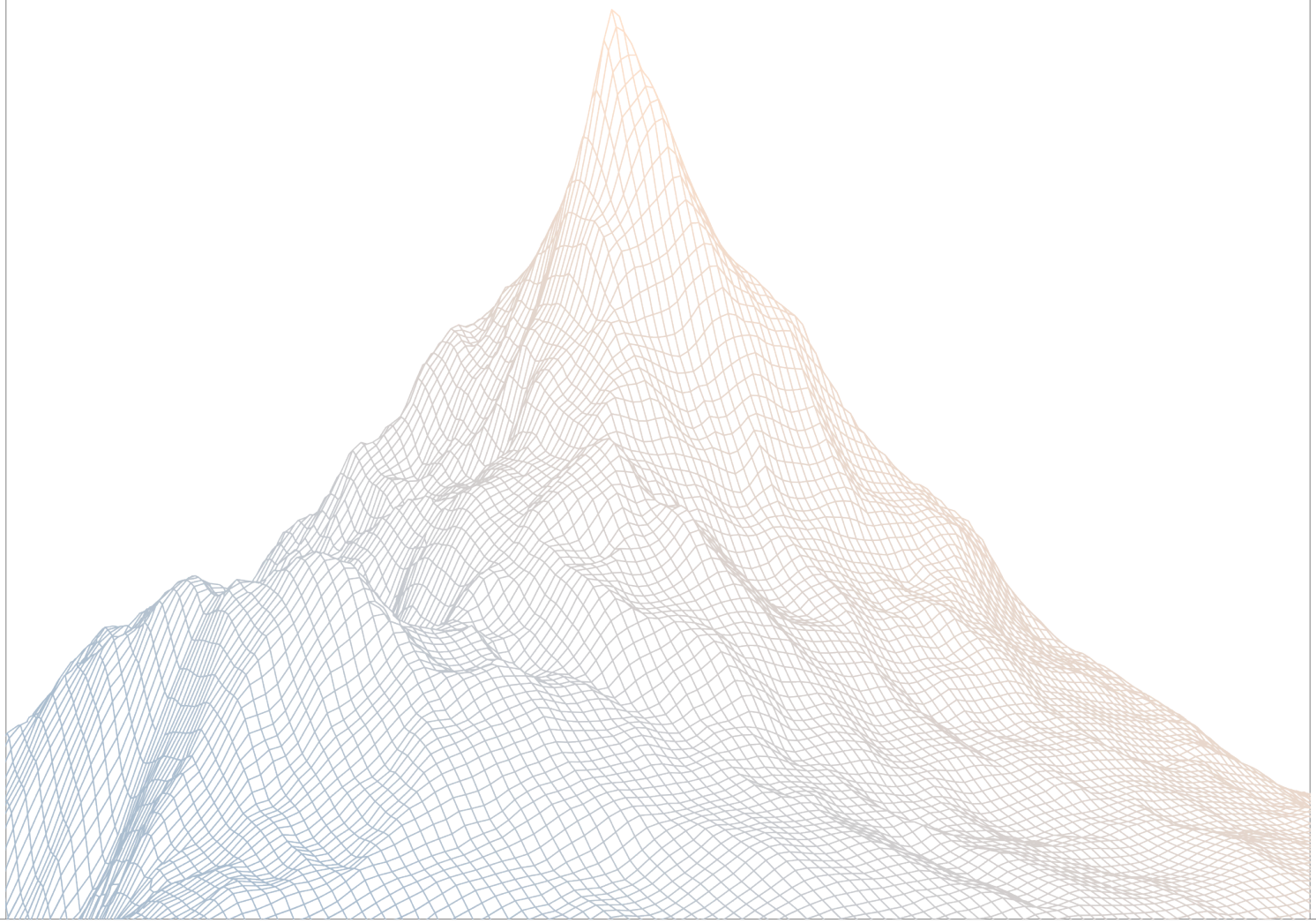
Zenith

Kite AI

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: March 12th to March 20th, 2026

AUDITED BY: ast3ros
ether_sky

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About Kite AI 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 5

4 Findings 7

4.1 Critical Risk 8

4.2 Medium Risk 12

4.3 Low Risk 23

4.4 Informational 31

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Kite AI

Kite enables AI agents to autonomously transact at scale with cryptographic safety and native x402 compatibility—solving the infrastructure crisis imprisoning the agent economy today.

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts-external
Repository	https://github.com/gokite-ai/contracts-external.git
Commit Hash	`6e2d086a11d53e8b43368de8f7f9ed296ee64596`

Files	`FixedAPRRewardCalculator.sol`
	`KiteStakingManager.sol`
	`RewardVault.sol`
	`StakingManager.sol`

Target	Mitigation Review
Repository	https://github.com/gokite-ai/contracts-external.git
Commit Hash	`ff0118bd617d187832cac15d0d6db8d393da7bfc`

Files	`FixedAPRRewardCalculator.sol`
	`KiteStakingManager.sol`
	`RewardVault.sol`
	`StakingManager.sol`

2.3 Audit Timeline

March 12, 2026	Audit start
March 20, 2026	Audit end
March 30, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	1
High Risk	0
Medium Risk	4
Low Risk	3
Informational	10
Total Issues	**18**

3

Findings Summary

ID	Description	Status
C-1	Reentrancy in forceInitiateDelegatorRemoval enables full drain of staked funds and rewards	Resolved
M-1	Blocking delegator exits via malicious reward recipient	Resolved
M-2	The delegator should be removed if the validator is in a pending removal state in the completeDelegatorRegistration function	Resolved
M-3	PendingAdded delegations can bypass the one-period churn lock when the validator has completed	Resolved
M-4	Rounding loss in value-to-weight conversion causes permanent loss of staked tokens	Resolved
L-1	Validators cannot claim rewards in some cases	Acknowledged
L-2	Delegators cannot claim rewards in some cases	Acknowledged
L-3	Active reward claims revert on transfer failure instead of storing in redeemable mappings	Acknowledged
I-1	Reward-related values should be updated when initiating validator removal	Acknowledged
I-2	PoS validators can be overwritten during the migration from V1 to V2	Acknowledged
I-3	The delegator's staking amount should meet a minimum requirement	Acknowledged
I-4	There is no need to send a warp message if the validator's weight is already up to date	Resolved
I-5	The uptime may be outdated when initiating delegator removal	Acknowledged
I-6	Incorrect value emitted in ValidatorRewardClaimed	Resolved

ID	Description	Status
I-7	No mechanism to reduce voting weight of misbehaving validator	Acknowledged
I-8	Unused import in StakingManager	Resolved
I-9	There are duplicated code parts	Resolved
I-10	There are some unused errors	Resolved

4

Findings

4.1 Critical Risk

A total of 1 critical risk findings were identified.

[C-1] Reentrancy in forceInitiateDelegatorRemoval enables full drain of staked funds and rewards

SEVERITY: Critical	IMPACT: High
STATUS: Resolved	LIKELIHOOD: High

Target

- StakingManager.sol#L1425
- StakingManager.sol#L1817

Description:

The `forceInitiateDelegatorRemoval` lacks the `nonReentrant` modifier.

```
    function forceInitiateDelegatorRemoval(
        bytes32 delegationID,
        bool includeUptimeProof,
        uint32 messageIndex
▷ ) external {
        // Ignore the return value here to force end delegation, regardless
    of possible missed rewards
        _initiateDelegatorRemoval(
            delegationID,
            includeUptimeProof,
            messageIndex
        );
    }
```

When the validator's status is `Completed`, `_initiateDelegatorRemoval` calls `_completeDelegatorRemoval`, which triggers native-token transfers before updating the delegator's state:

```
    function _initiateDelegatorRemoval(
```

```
        bytes32 delegationID,
        bool includeUptimeProof,
        uint32 messageIndex
    ) internal returns (bool) {
        ...
        } else if (validator.status == ValidatorStatus.Completed) {
            _calculateAndSetDelegationReward(
                delegator,
                rewardRecipient,
                delegationID
            );
▷           _completeDelegatorRemoval(delegationID);
            // If the validator has completed, then no further uptimes may be
    submitted, so we always
            // end the delegation.
            return true;
        ...
    }
```

Inside `_completeDelegatorRemoval`, it calls `_completeDelegatorRemoval`. And `_reward(rewardRecipient, delegationRewards)` sends native tokens via `sendValue` to the attacker-controlled `rewardRecipient`.

```
    function _withdrawDelegationRewards(
        address rewardRecipient,
        bytes32 delegationID,
        bytes32 validationID
    ) internal returns (uint256, uint256) {
        StakingManagerStorage storage $ = _getStakingManagerStorage();

        uint256 delegationRewards;
        uint256 validatorFees;

        uint256 rewards = $._redeemableDelegatorRewards[delegationID];

        if (rewards > 0) {
            validatorFees =
                (rewards *
                    $._posValidatorInfo[validationID].delegationFeeBips) /
                BIPS_CONVERSION_FACTOR;

            // Reward the remaining tokens to the delegator.
            delegationRewards = rewards - validatorFees;
▷           bool success = _reward(rewardRecipient, delegationRewards);

            // Only clear rewards and allocate validator fees if distribution
```

```
succeeded
        // If failed, rewards remain claimable via claimDelegatorRewards
        if (success) {
            delete $._redeemableDelegatorRewards[delegationID];
            delete $._delegatorRewardRecipients[delegationID];

        ...
}
```

When `_reward` is first called, the delegator's status is still `Active` and weight is unchanged. An attacker-controlled `rewardRecipient` contract can re-enter `forceInitiateDelegatorRemoval` with the same `delegationID`, repeating the full flow: reward recalculation, reward payout, and stake unlock. Each re-entrant iteration drains both `RewardVault` (via `_reward`) and `StakingManager` (via `_unlock`), continuing until the contracts are emptied.

```
function _completeDelegatorRemoval(bytes32 delegationID) internal {
    ...
    (
        uint256 delegationRewards,
        uint256 validatorFees
    ) = _withdrawDelegationRewards(
            rewardRecipient,
            delegationID,
            validationID
        );

    if (
        delegationRewards > 0 ||
        $._redeemableDelegatorRewards[delegationID] == 0
    ) {
        // Success: clean up all data
        delete $._delegatorStakes[delegationID];
    } else {
        // Failed: preserve essential data, mark as completed
        $._delegatorStakes[delegationID].status
= DelegatorStatus.Unknown;
        // Keep: validationID, owner (for permission check in
claimDelegatorRewards)
        // Clear unnecessary fields to save gas
        $._delegatorStakes[delegationID].weight = 0;
        $._delegatorStakes[delegationID].startTime = 0;
        $._delegatorStakes[delegationID].startingNonce = 0;
        $._delegatorStakes[delegationID].endingNonce = 0;
        $._delegatorStakes[delegationID].lastRewardClaimTime = 0;
        $._delegatorStakes[delegationID].lastClaimUptimeSeconds = 0;
```

```
        }

        // Unlock the delegator's stake.
▷       uint256 stakeAmount = weightToValue(delegator.weight);
▷       _unlock(delegator.owner, stakeAmount);

        emit CompletedDelegatorRemoval(
            delegationID,
            validationID,
            stakeAmount,
            delegationRewards,
            validatorFees
        );
    }
```

Recommendations:

- Add the `nonReentrant` modifier to `forceInitiateDelegatorRemoval`
- refactor all functions to follow the checks-effects-interactions pattern. Updating states before sending native tokens.

Kite AI: Resolved with `@17fbda11ff ...`

Zenith: Verified.

4.2 Medium Risk

A total of 4 medium risk findings were identified.

[M-1] Blocking delegator exits via malicious reward recipient

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingManager.sol#L1520
- StakingManager.sol#L863
- StakingManager.sol#L1773
- RewardVault.sol#L118

Description:

A malicious validator can permanently lock all delegator stakes by setting their reward recipient to a contract that rejects native token transfers, preventing `completeValidatorRemoval` from ever succeeding.

The attack works as follows. When a validator's removal is initiated, its status transitions to `PendingRemoved`. Delegators cannot initiate their own removal while the validator is in this state. `_initiateDelegatorRemoval` only permits removal when the validator is `Active` or `Completed`, and reverts otherwise:

```
    function _initiateDelegatorRemoval(
        bytes32 delegationID,
        bool includeUptimeProof,
        uint32 messageIndex
    ) internal returns (bool) {
        ...
        if (validator.status == ValidatorStatus.Active) {
            ...
        } else if (validator.status == ValidatorStatus.Completed) {
            ...
        } else {
▷           revert InvalidValidatorStatus(validator.status);
```

```
        }
    }
```

However, a malicious validator can always revert when calling `completeValidatorRemoval`.

```
    function completeValidatorRemoval(
        uint32 messageIndex
    ) external nonReentrant returns (bytes32) {
        ...

        // The validator can either be Completed or Invalidated here.  We
    only grant rewards for Completed.
        if (validator.status == ValidatorStatus.Completed) {
▷           _withdrawValidationRewards(rewardRecipient, validationID);
        } else {
            // If invalidated, no rewards are given
            rewards = 0;
        }

        ...
    }
```

The only path to unblock delegators is to complete the validator's removal so its status advances to `Completed`. However, `completeValidatorRemoval` calls `_withdrawValidationRewards`, which in turn calls `_reward`:

```
    function _withdrawValidationRewards(
        address rewardRecipient,
        bytes32 validationID
    ) internal {
        StakingManagerStorage storage $ = _getStakingManagerStorage();

        uint256 rewards = $._redeemableValidatorRewards[validationID];
        if (rewards == 0) {
            return;
        }

▷       bool success = _reward(rewardRecipient, rewards);

        // Only clear rewards if distribution succeeded
        // If failed, rewards remain claimable via claimValidatorRewards
        if (success) {
            delete $._redeemableValidatorRewards[validationID];
            emit ValidatorRewardClaimed(validationID, rewardRecipient,
    rewards);
```

```
        }
    }
```

While `_reward` handles some failure modes gracefully, the actual `vault.distributeReward` call is not wrapped in a try-catch. Inside `RewardVault.distributeReward`, the reward is sent via `sendValue`, which reverts if the recipient rejects the transfer:

```
function distributeReward(address to, uint256 amount) external {
        if (msg.sender ≠ stakingManager) {
            revert UnauthorizedCaller(msg.sender);
        }
        if (to == address(0)) {
            revert ZeroAddress();
        }
        if (amount > address(this).balance) {
            revert InsufficientBalance(amount, address(this).balance);
        }
▷      payable(to).sendValue(amount);
        emit RewardDistributed(to, amount);
    }
```

Because `completeValidatorRemoval` can never succeed, the validator is permanently stuck in the `PendingRemoved` state. The delegators cannot initiate removal. Their funds are locked in the KiteStakingManager indefinitely.

The validator can attract delegations and change the reward recipient later by calling `changeValidatorRewardRecipient` to perform the lock. With the default `maximumStakeMultiplier` of 20x, an attacker can lock up to 20 times their own stake in delegator funds. It also violates the invariants: `completeValidatorRemoval and completeDelegatorRemoval must always return the user's full principal stake, even if reward distribution fails.`

Recommendations:

Wrap the `vault.distributeReward` call in a try-catch so that a transfer failure returns `false` instead of reverting.

Kite AI: Resolved with `@7250a862b3 ...`

Zenith: Verified.

[M-2] The delegator should be removed if the validator is in a pending removal state in the completeDelegatorRegistration function

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- StakingManager.sol

Description:

When `validator removal` is initiated, the `validator`'s `uptime` is updated.

```
function _initiatePoSValidatorRemoval(
    bytes32 validationID,
    bool includeUptimeProof,
    uint32 messageIndex
) internal {
    // Uptime proofs include the absolute number of seconds the validator has
    been active.
    uint64 uptimeSeconds;
    if (includeUptimeProof) {
        uptimeSeconds = _updateUptime(validationID, messageIndex);
    } else {
        uptimeSeconds = $._posValidatorInfo[validationID].uptimeSeconds;
    }
```

Although this value may be outdated, it becomes fixed for the `validator` and all active `delegators` associated with it. `Uptime proofs` can only be submitted while the `validator` is `active`.

```
function submitUptimeProof(
    bytes32 validationID,
    uint32 messageIndex
) external {
    if (status ≠ ValidatorStatus.Active) {
        revert InvalidValidatorStatus(status);
    }
```

```
}
```

However, the `validator`'s `uptime` can still be updated in the following scenario:

- `Delegators` are allowed to initiate `registration` only when the `validator` is `active`.
- Once `active`, the `validator` can transition only to `PendingRemoved` or `Completed`.
- Suppose the `validator` initiates `removal` and moves to `PendingRemoved`.
- Later, a `delegator` in `PendingAdded` status calls `completeDelegatorRegistration` with a newer `uptime proof`.

```
function completeDelegatorRegistration(
    bytes32 delegationID,
    uint32 messageIndex,
    uint32 uptimeMessageIndex
) external {
    if (delegator.status ≠ DelegatorStatus.PendingAdded) {
        revert InvalidDelegatorStatus(delegator.status);
    }

    if (validator.status == ValidatorStatus.Completed) {
        return _completeDelegatorRemoval(delegationID);
    }

    uint64 currentUptime = _updateUptime(validationID, uptimeMessageIndex);
```

At this point, `rewards` have already been calculated using the previous `uptime` for that `validator`, and since the `validator` is no longer `active`, submitting a new `uptime proof` should not be allowed. However, the `uptime` gets updated again, and the `delegator` is registered to a `non-active validator`.

This leads to two issues:

- Existing `active delegators` benefit from the newly updated `uptime`.
- The newly registered `delegator` should immediately exit, as they are not eligible for any `rewards`.

Recommendations:

```
function completeDelegatorRegistration(
    bytes32 delegationID,
    uint32 messageIndex,
    uint32 uptimeMessageIndex
) external {
    if (delegator.status ≠ DelegatorStatus.PendingAdded) {
```

```
        revert InvalidDelegatorStatus(delegator.status);
    }

    if (validator.status == ValidatorStatus.Completed) {

    if (validator.status == ValidatorStatus.Completed || validator.status ==
          ValidatorStatus.PendingRemoved) {
        return _completeDelegatorRemoval(delegationID);
    }

    uint64 currentUptime = _updateUptime(validationID, uptimeMessageIndex);
```

Kite AI: Resolved with `@f38dca2c44 ...`

Zenith: Verified.

[M-3] PendingAdded delegations can bypass the one-period churn lock when the validator has completed

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- StakingManager.sol#L1681-L1686

Description:

`_initiateDelegatorRegistration` triggers a validator weight update, consuming churn quota for the current period:

```
function _initiateDelegatorRegistration(
    bytes32 validationID,
    address delegatorAddress,
    uint256 delegationAmount,
    address rewardRecipient
) internal returns (bytes32) {
    ...

    (uint64 nonce, bytes32 messageID) = $
        ._manager
       .initiateValidatorWeightUpdate(validationID, newValidatorWeight);

    ...
}
```

When `completeDelegatorRegistration` is called against a `Completed` validator, it short-circuits directly to `_completeDelegatorRemoval` without setting `delegator.startTime`:

```
function completeDelegatorRegistration(
    bytes32 delegationID,
    uint32 messageIndex,
    uint32 uptimeMessageIndex
) external {
    ...
```

```
        // In the case where the validator has completed its validation
    period, we can no
        // longer stake and should move our status directly to completed and
    return the stake.
        if (validator.status == ValidatorStatus.Completed) {
▷           return _completeDelegatorRemoval(delegationID);
        }

        ...

        $._delegatorStakes[delegationID].status = DelegatorStatus.Active;
▷      $._delegatorStakes[delegationID].startTime = uint64(block.timestamp);
        ...
    }
```

Because `startTime` is left as 0, the minimum-duration check in `_completeDelegatorRemoval` is bypassed (since `block.timestamp ≥ 0 + churnPeriodSeconds` is always true), returning the stake immediately.

```
    function _completeDelegatorRemoval(bytes32 delegationID) internal {
        ...
        if (
            block.timestamp <
▷           delegator.startTime + $._manager.getChurnPeriodSeconds()
        ) {
            revert MinStakeDurationNotPassed(uint64(block.timestamp));
        }

        ...
    }
```

Therefore, the bug exists whenever a delegator stays `PendingAdded` and the validator reaches `Completed`.

Consider a scenario:

- An attacker front-runs a validator's `initiateValidatorRemoval` call with `initiateDelegatorRegistration` on the same validator. This consumes churn quota and records the delegator as `PendingAdded` with `startTime = 0`.
- The validator removal completes, transitioning it to `Completed`.
- The attacker calls `completeDelegatorRegistration`. The `Completed` branch skips activation and immediately calls `_completeDelegatorRemoval`, returning the stake with no lock enforced.
- The attacker repeats steps 1—3 within the same churn window with different validators, recycling the same capital to exhaust the churn budget multiple times.

This violates the protocol invariant: `Churn Limits: minimumStakeDuration ≥ churnPeriodSeconds must always hold to prevent a delegator from staking twice in the same churn period.` An attacker can cheaply exhaust the churn budget and temporarily DoS legitimate churn-limited operations.

Recommendations:

Enforce the one churn period lock consistently regardless of validator state by storing a separate churn-lock timestamp at delegation initiation.

Kite AI: Resolved with `@c1b80ca850 ...`

Zenith: Verified.

[M-4] Rounding loss in value-to-weight conversion causes permanent loss of staked tokens

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingManager.sol#L1016-L1032

Description:

`valueToWeight` truncates via integer division, discarding any remainder (`value % _weightToValueFactor`).

```
function valueToWeight(uint256 value) public view returns (uint64) {
    uint256 weight = value /
        _getStakingManagerStorage()._weightToValueFactor;
    if (weight == 0 || weight > type(uint64).max) {
        revert InvalidStakeAmount(value);
    }
    return uint64(weight);
}
```

On withdrawal, `weightToValue` reconstructs the amount from the truncated weight, so the remainder is never returned. The difference is locked in the contract permanently with no recovery path.

```
function weightToValue(uint64 weight) public view returns (uint256) {
    return
        uint256(weight) * _getStakingManagerStorage()._weightToValueFactor;
}
```

The same issue exists for delegations. That directly breaks `completeValidatorRemoval and completeDelegatorRemoval must always return the user's full principal stake` invariants.

Recommendations:

Reject any amount that is not an exact multiple of `_weightToValueFactor` before locking.

Kite AI: Resolved with `@2f378f54c9 ...`

Zenith: Verified.

4.3 Low Risk

A total of 3 low risk findings were identified.

[L-1] Validators cannot claim rewards in some cases

SEVERITY: Low	IMPACT: High
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingManager.sol

Description:

Suppose a `validator` is about to be removed using `initiateValidatorRemoval`. During this function, the most recent active `uptime` may be updated and the `validator` could claim the `rewards` based on this updated new `uptime`.

However, if `claimValidatorRewardsFor` is executed **in the same block before** `initiateValidatorRemoval` (either due to front-running or accidental ordering), the `validator`'s `lastRewardClaimTime` will be set to the current `block.timestamp`.

```
function _claimActiveValidatorRewards(
    bytes32 validationID,
    address rewardRecipient,
    Validator memory validator,
    bool includeUptimeProof,
    uint32 messageIndex
) internal returns (uint256 reward) {
    (uint64 currentUptime, uint64 currentTime) = _getUpdatedUptime(
        validationID,
        validator.status,
        includeUptimeProof,
        messageIndex
    );

    $._posValidatorInfo[validationID].lastRewardClaimTime = currentTime;
```

After that, when `initiateValidatorRemoval` is executed, the `validator` cannot receive

additional `rewards` based on the newly updated `uptime` because `lastRewardClaimTime` has already been set to the current `block.timestamp`.

```
function _initiatePoSValidatorRemoval(
    bytes32 validationID,
    bool includeUptimeProof,
    uint32 messageIndex
) internal {
    // Uptime proofs include the absolute number of seconds the validator has
    been active.
    uint64 uptimeSeconds;
    if (includeUptimeProof) {
        uptimeSeconds = _updateUptime(validationID, messageIndex);
    } else {
        uptimeSeconds = $._posValidatorInfo[validationID].uptimeSeconds;
    }

    // Calculate remaining reward from last claim (or start) to end
    PoSValidatorInfo storage posInfo = $._posValidatorInfo[validationID];
    uint64 lastClaimTime = posInfo.lastRewardClaimTime;
    uint64 lastClaimUptime = posInfo.lastClaimUptimeSeconds;

    // If never claimed before, use validator start time
    if (lastClaimTime == 0) {
        lastClaimTime = validator.startTime;
        lastClaimUptime = 0;
    }

    uint256 reward = $._rewardCalculator.calculateIncrementalReward({
        stakeAmount: weightToValue(validator.startingWeight),
        lastClaimTime: lastClaimTime,
        currentTime: validator.endTime, // lastClaimTime == currentTime
        lastClaimUptimeSeconds: lastClaimUptime,
        currentUptimeSeconds: uptimeSeconds,
        validatorStartTime: validator.startTime
    });
```

Recommendations:

```
function _claimActiveValidatorRewards(
    bytes32 validationID,
    address rewardRecipient,
    Validator memory validator,
    bool includeUptimeProof,
```

```
    uint32 messageIndex
) internal returns (uint256 reward) {
    (uint64 currentUptime, uint64 currentTime) = _getUpdatedUptime(
        validationID,
        validator.status,
        includeUptimeProof,
        messageIndex
    );


    if ($._posValidatorInfo[validationID].status == ValidatorStatus.Active) {
        currentTime = currentTime - 1;
    }

    $._posValidatorInfo[validationID].lastRewardClaimTime = currentTime;
```

Kite AI: Acknowledged. This is the validator-side equivalent of the same-block reward-accounting edge case caused by ordering between a permissionless active reward claim and validator-removal settlement when a fresher uptime proof is only incorporated during removal. It does not lock funds or prevent reward claims generally; rather, it can strand only a marginal slice of validator rewards at validator exit under a narrow same-block ordering condition.

[L-2] Delegators cannot claim rewards in some cases

SEVERITY: Low	IMPACT: High
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingManager.sol

Description:

Suppose a `validator` is about to be removed using `initiateValidatorRemoval`. During this function, the most recent active `uptime` may be updated. Therefore, all active `delegators` should receive `rewards` based on this updated `uptime`.

However, if `claimDelegatorRewardsFor` is executed for an active `delegator` **in the same block before** `initiateValidatorRemoval` (either due to front-running or accidental ordering), the `delegator`'s `lastRewardClaimTime` will be set to the current `block.timestamp`, and the `rewards` will be calculated using the **old uptime**.

```
function _claimActiveDelegatorRewards(
    bytes32 delegationID,
    Delegator storage delegator,
    bool includeUptimeProof,
    uint32 messageIndex
) internal returns (uint256 reward) {
    uint256 grossReward = $._rewardCalculator.calculateIncrementalReward({
        stakeAmount: weightToValue(delegator.weight),
        lastClaimTime: lastClaimTime,
        currentTime: currentTime,
        lastClaimUptimeSeconds: lastClaimUptime,
        currentUptimeSeconds: currentUptime,
        validatorStartTime: validator.startTime
    });
    delegator.lastRewardClaimTime = currentTime;
    delegator.lastClaimUptimeSeconds = currentUptime;
}
```

Afterwards, when `initiateValidatorRemoval` executes, the `validator`'s active `uptime` is updated and the `validator`'s `endTime` is also set to the current `block.timestamp`.

```
function _initiateValidatorRemoval(
    bytes32 validationID
) internal virtual override {
    // Update the validator status to pending removal.
    // They are not removed from the active validators mapping until the
    P-Chain acknowledges the removal.
    validator.status = ValidatorStatus.PendingRemoved;

    // Set the end time of the validation period, since it is no longer known
    to be an active validator
    // on the P-Chain.
    validator.endTime = uint64(block.timestamp);
```

As a result, those `delegators` cannot `claim` the additional `rewards` corresponding to the newly updated `uptime`. This happens because `calculateIncrementalReward` returns `0` when `currentTime == lastClaimTime`, even if `currentUptimeSeconds` is greater than `lastClaimUptimeSeconds`.

```
function calculateIncrementalReward(
    uint256 stakeAmount,
    uint64 lastClaimTime,
    uint64 currentTime,
    uint64 lastClaimUptimeSeconds,
    uint64 currentUptimeSeconds,
    uint64 validatorStartTime
) external view returns (uint256 reward) {
    // Silence unused variable warning
    validatorStartTime;

    // Calculate the time period for this claim
    uint64 periodDuration = currentTime - lastClaimTime;
    if (periodDuration == 0) {
        return 0;
    }

    // Calculate uptime for this period
    uint64 periodUptimeSeconds = currentUptimeSeconds -
        lastClaimUptimeSeconds;

    // Linear reward: proportional to actual uptime
    // reward = stakeAmount * rewardBasisPoints * periodUptimeSeconds /
    (SECONDS_IN_YEAR * BIPS_CONVERSION_FACTOR)
    return
        (stakeAmount * rewardBasisPoints * periodUptimeSeconds) /
        (SECONDS_IN_YEAR * BIPS_CONVERSION_FACTOR);
```

```
}
```

Recommendations:

```
function _claimActiveDelegatorRewards(
    bytes32 delegationID,
    Delegator storage delegator,
    bool includeUptimeProof,
    uint32 messageIndex
) internal returns (uint256 reward) {
    (uint64 currentUptime, uint64 currentTime) = _getUpdatedUptime(
        validationID,
        validator.status,
        includeUptimeProof,
        messageIndex
    );

    if (
        validator.status == ValidatorStatus.PendingRemoved ||
        validator.status == ValidatorStatus.Completed
    ) {
        currentTime = validator.endTime;
    }

    ...

    if (validator.status == ValidatorStatus.Active) {
        currentTime = currentTime - 1;
    }

    delegator.lastRewardClaimTime = currentTime;
    delegator.lastClaimUptimeSeconds = currentUptime;
}
```

Kite AI: Acknowledged. This is a same-block reward-accounting edge case caused by ordering between a permissionless active delegation claim and validator-removal settlement when a fresher uptime proof is only incorporated during removal. It does not lock funds or prevent reward claims generally; rather, it can strand only a marginal slice of delegator rewards at validator exit under a narrow ordering condition.

[L-3] Active reward claims revert on transfer failure instead of storing in redeemable mappings

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingManager.sol#L465-L468
- FixedAPRRewardCalculator.sol#L64-L79

Description:

The project specifies a "Reward Fallback Safety" invariant: `_reward` failure must never block operations, and failed rewards must be preserved in `_redeemableValidatorRewards` / `_redeemableDelegatorRewards` for later claiming.

However, both `_claimActiveValidatorRewards` and `_claimActiveDelegatorRewards` functions violate this invariant:

```
    function _claimActiveValidatorRewards(
        bytes32 validationID,
        address rewardRecipient,
        Validator memory validator,
        bool includeUptimeProof,
        uint32 messageIndex
    ) internal returns (uint256 reward) {
        ...

        // Transfer rewards after state update
        bool success = _reward(rewardRecipient, reward);
▷       if (!success) {
            revert RewardClaimFailed();
        }

        ...
    }
```

It's an issue because the `FixedAPRRewardCalculator.setRewardBasisPoints` applies the new rate retroactively to all unclaimed periods.

```
    function setRewardBasisPoints(uint64 newRewardBasisPoints)
    external onlyOwner {
        if (newRewardBasisPoints == 0) {
            revert ZeroRewardBasisPoints();
        }
        if (newRewardBasisPoints > MAX_REWARD_BASIS_POINTS) {
            revert RewardBasisPointsExceedsMax(newRewardBasisPoints,
    MAX_REWARD_BASIS_POINTS);
        }
        uint64 oldBasisPoints = rewardBasisPoints;
▷      rewardBasisPoints = newRewardBasisPoints;
        emit RewardBasisPointsUpdated(oldBasisPoints, newRewardBasisPoints);
    }
```

If the `RewardVault` temporarily lacks funds and causes `_reward` to return `false` around the time the owner lowers the reward rate, validators and delegators cannot checkpoint their accrued rewards at the previous higher rate. Once the rate decreases, the entire unclaimed period is recalculated at the lower rate, resulting in a loss of earned rewards.

Recommendations:

Instead of reverting when `_reward` fails, store the computed reward in the corresponding redeemable mapping (`_redeemableValidatorRewards` / `_redeemableDelegatorRewards`) and allow the state updates to persist.

Kite AI: Acknowledged. We agree this is a limitation of the active-claim path. However, that invariant is specifically about ensuring reward-transfer failures do not block validator/delegator exit or stake unlocking during removal flows, not about guaranteeing fallback storage for failed active claims. In practice, reward-rate updates are preceded by a batch reward-settlement process, which mitigates the scenario described here.

4.4 Informational

A total of 10 informational findings were identified.

[I-1] Reward-related values should be updated when initiating validator removal

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingManager.sol

Description:

The `lastRewardClaimTime` and `lastClaimUptimeSeconds` are not updated, even though `rewards` are calculated based on the current timestamps.

```
function _initiatePoSValidatorRemoval(
    bytes32 validationID,
    bool includeUptimeProof,
    uint32 messageIndex
) internal {
    if (includeUptimeProof) {
        uptimeSeconds = _updateUptime(validationID, messageIndex);
    } else {
        uptimeSeconds = $._posValidatorInfo[validationID].uptimeSeconds;
    }

    // Calculate remaining reward from last claim (or start) to end
    PoSValidatorInfo storage posInfo = $._posValidatorInfo[validationID];
    uint64 lastClaimTime = posInfo.lastRewardClaimTime;
    uint64 lastClaimUptime = posInfo.lastClaimUptimeSeconds;

    uint256 reward = $._rewardCalculator.calculateIncrementalReward({
        stakeAmount: weightToValue(validator.startingWeight),
        lastClaimTime: lastClaimTime,
        currentTime: validator.endTime,
```

```
        lastClaimUptimeSeconds: lastClaimUptime,
        currentUptimeSeconds: uptimeSeconds,
        validatorStartTime: validator.startTime
    });

    $._redeemableValidatorRewards[validationID] += reward;
}
```

Recommendations:

```
function _initiatePoSValidatorRemoval(
    bytes32 validationID,
    bool includeUptimeProof,
    uint32 messageIndex
) internal {
    $._redeemableValidatorRewards[validationID] += reward;

    posInfo.lastRewardClaimTime = validator.endTime;
    posInfo.lastClaimUptimeSeconds = uptimeSeconds;
}
```

Kite AI: Acknowledged. Once removal is initiated, remaining rewards are checkpointed into _redeemableValidatorRewards, so later non-active reward flows no longer rely on lastRewardClaimTime or lastClaimUptimeSeconds; we therefore do not consider a code change necessary.

[I-2] PoS validators can be overwritten during the migration from V1 to V2

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingManager.sol

Description:

`PoS validators` can be registered using the `_initiateValidatorRegistration` function. If a `validationId` already has a `_validationPeriodsLegacy` value, it may be overwritten during the migration process.

```
function migrateFromV1(
    bytes32 validationID,
    uint32 receivedNonce
) external {
    ValidatorManagerStorage storage $ = _getValidatorManagerStorage();
    ValidatorLegacy storage legacy = $._validationPeriodsLegacy[
        validationID
    ];
    if (legacy.status == ValidatorStatus.Unknown) {
        revert InvalidValidationID(validationID);
    }
    if (receivedNonce > legacy.messageNonce) {
        revert InvalidNonce(receivedNonce);
    }

    $._validationPeriods[validationID] = Validator({
        status: legacy.status,
        nodeID: legacy.nodeID,
        startingWeight: legacy.startingWeight,
        sentNonce: legacy.messageNonce,
        receivedNonce: receivedNonce,
        weight: legacy.weight,
        startTime: legacy.startedAt,
        endTime: legacy.endedAt
    });
```

```
    // Set the legacy status to unknown to disallow future migrations.
    $._validationPeriodsLegacy[validationID].status = ValidatorStatus
        .Unknown;
}
```

Recommendations:

If the `node IDs` of `V1 validators` are already registered in `_registeredValidators`, this issue will not occur. However, if they are not, it is worth verifying whether `_validationPeriodsLegacy` already contains a value for a registered `PoS validator` - even though the likelihood is low.

Kite AI: Acknowledged. This migration path is inherited from the upstream Avalanche implementation, but it is not used in our deployment. Our system was deployed directly on the V2 version from the beginning and does not rely on migrateFromV1, so the reported overwrite scenario is not applicable to our setup.

[I-3] The delegator's staking amount should meet a minimum requirement

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingManager.sol

Description:

There is a minimum `staking` requirement for `validators`.

```
function _initiateValidatorRegistration(
    bytes memory nodeID,
    bytes memory blsPublicKey,
    PChainOwner memory remainingBalanceOwner,
    PChainOwner memory disableOwner,
    uint16 delegationFeeBips,
    uint64 minStakeDuration,
    uint256 stakeAmount,
    address rewardRecipient
) internal virtual returns (bytes32) {
    // Ensure the weight is within the valid range.
    if (
        stakeAmount < $._minimumStakeAmount ||
        stakeAmount > $._maximumStakeAmount
    ) {
        revert InvalidStakeAmount(stakeAmount);
    }
```

However, `delegators` can `stake` any amount as long as it is not less than `_weightToValueFactor`. To prevent potential spamming, a stricter minimum `staking` requirement for `delegators` should be enforced.

Recommendations:

Add a minimum amount check for `delegators`, similar to the one used for `validators`.

Kite AI: Acknowledged. The validator path enforces an explicit minimumStakeAmount, while the delegator path does not; instead, delegator stake is constrained indirectly by valueToWeight(...) and the validator-level maximum stake multiplier.

[I-4] There is no need to send a warp message if the validator's weight is already up to date

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingManager.sol

Description:

Anyone can trigger a `validator weight update` by sending a `warp message` through the `resendUpdateDelegator` function.

```
function resendUpdateDelegator(bytes32 delegationID) external {
    if (validator.sentNonce == 0) {
        // Should be unreachable.
        revert InvalidDelegationID(delegationID);
    }

    // Submit the message to the Warp precompile.
    WARP_MESSENGER.sendWarpMessage(
        ValidatorMessages.packL1ValidatorWeightMessage(
            delegator.validationID,
            validator.sentNonce,
            validator.weight
        )
    );
}
```

However, if `sentNonce` and `receivedNonce` are equal, the `validator`'s `weight` is already up to date, and sending the message is unnecessary in order to prevent spamming.

Recommendations:

```
function resendUpdateDelegator(bytes32 delegationID) external {
    if (validator.sentNonce == 0) {
        // Should be unreachable.
        revert InvalidDelegationID(delegationID);
    }

    if (validator.sentNonce == validator.receivedNonce) {
        return;
    }

    // Submit the message to the Warp precompile.
    WARP_MESSENGER.sendWarpMessage(
        ValidatorMessages.packL1ValidatorWeightMessage(
            delegator.validationID,
            validator.sentNonce,
            validator.weight
        )
    );
}
```

Kite AI: Resolved with `@9e31adc24f ...`

Zenith: Verified.

[I-5] The uptime may be outdated when initiating delegator removal

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingManager.sol

Description:

When initiating `delegator removal`, the `delegator owner` can update the `validator`'s `uptime` to calculate accurate `rewards`.

```
function _initiateDelegatorRemoval(
    bytes32 delegationID,
    bool includeUptimeProof,
    uint32 messageIndex
) internal returns (bool) {
```

However, there is no guarantee that this value reflects the most up-to-date or correct `uptime`.

Recommendations:

Add comments similar to those in the `completeDelegatorRegistration` function

```
// Note: The stored uptimeSeconds may not be the absolute latest value, but
    we mitigate
// this through: (1) periodic uptime proof submissions (e.g., per 15 mins)
    for all validators,
// and (2) uptime updates triggered by reward claims. This keeps the
    deviation minimal
// and acceptable for reward calculations.
```

Kite AI: Acknowledged. The uptime used here may not be the absolute latest value, but this is already operationally mitigated by periodic uptime-proof submission and claim-triggered updates. We do not consider a code change necessary here.

[I-6] Incorrect value emitted in ValidatorRewardClaimed

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingManager.sol#L473
- StakingManager.sol#L1779

Description:

In `_claimActiveValidatorRewards`, the total reward distributed to the recipient is `stakingReward + delegationFees`, but the `ValidatorRewardClaimed` event only emits `stakingReward`:

```
    function _claimActiveValidatorRewards(
        bytes32 validationID,
        address rewardRecipient,
        Validator memory validator,
        bool includeUptimeProof,
        uint32 messageIndex
    ) internal returns (uint256 reward) {
        ...
        // Total reward = staking reward + delegation fees
▷       reward = stakingReward + delegationFees;
        ...

        emit ValidatorRewardClaimed(
            validationID,
            rewardRecipient,
▷           stakingReward // delegationFees excluded
        );
    }
```

This is inconsistent with `_withdrawValidationRewards` and `claimValidatorRewards`, which correctly emits the full reward amount:

```
    function _withdrawValidationRewards(
        address rewardRecipient,
```

```
        bytes32 validationID
    ) internal {
        ...
        if (success) {
            delete $._redeemableValidatorRewards[validationID];
▷           emit ValidatorRewardClaimed(validationID, rewardRecipient,
    rewards); // full amount
        }
    }
```

Recommendations:

Pass the total reward (`stakingReward + delegationFees`) as the third argument to `ValidatorRewardClaimed` in `_claimActiveValidatorRewards`.

Kite AI: Resolved with `@06ddba97a0 ...`

Zenith: Verified.

[I-7] No mechanism to reduce voting weight of misbehaving validator

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingManager.sol

Description:

ACP 77 - Reinventing Subnets specifies:

If a validator is misbehaving, the L1 validators can collectively generate a BLS multisig to reduce the voting weight of a misbehaving validator.

https://github.com/avalanche-foundation/ACPs/tree/main/ACPs/77-reinventing-subnets#sidebar-l1-sovereignty

The current `StakingManager` implementation does not expose any on-chain mechanism to act on this. There is no function to slash stake, reduce voting weight, or otherwise penalize a validator that violates protocol rules.

Recommendations:

Consider implementing a slashing or weight-reduction mechanism that allows authorized parties to penalize misbehaving validators.

Kite AI: Acknowledged. In the current implementation, we do not expose an on-chain slashing or partial weight-reduction path for validator misconduct. What we do have today is full validator removal and admin force-removal.

[I-8] Unused import in StakingManager

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingManager.sol#L16

Description:

`OwnableUpgradeable` is imported in `StakingManager` but never used. The contract does not inherit from it, making the import dead code.

```
        pragma solidity 0.8.25;

        ...
▷     import {OwnableUpgradeable} from "@openzeppelin/contracts-
    upgradeable@5.0.2/access/OwnableUpgradeable.sol";


        abstract contract StakingManager is
```

Recommendations:

Remove the unused import.

Kite AI: Resolved with `@f0e3e4c1aa ...`

Zenith: Verified.

[I-9] There are duplicated code parts

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingManager.sol

Description:

The following parts are duplicated in the `completeDelegatorRemoval` function.

```
function completeDelegatorRemoval(
    bytes32 delegationID,
    uint32 messageIndex
) external nonReentrant {
@-> Validator memory validator = $._manager.getValidator(
        delegator.validationID
    );

    // We only expect an ICM message if we haven't received a weight update
    with a nonce greater than the delegation's ending nonce
    if (
@->     $._manager.getValidator(delegator.validationID).status ≠
        ValidatorStatus.Completed &&
        validator.receivedNonce < delegator.endingNonce
    ) {
```

Recommendations:

```
function completeDelegatorRemoval(
    bytes32 delegationID,
    uint32 messageIndex
) external nonReentrant {
    Validator memory validator = $._manager.getValidator(
        delegator.validationID
    );
```

```
// We only expect an ICM message if we haven't received a weight update
with a nonce greater than the delegation's ending nonce
if (
    $._manager.getValidator(delegator.validationID).status ≠
    validator.status ≠
    ValidatorStatus.Completed &&
    validator.receivedNonce < delegator.endingNonce
) {
```

Kite AI: Resolved with `@da136226c0 ...`

Zenith: Verified.

[I-10] There are some unused errors

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- RewardVault.sol

Description:

The `TransferFailed` error is not used in `RewardVault`.

```
/// @notice Error thrown when transfer fails
error TransferFailed();
```

Also, the `ValidatorIneligibleForRewards` and `DelegatorIneligibleForRewards` errors are not used in `StakingManager`.

```
error ValidatorIneligibleForRewards(bytes32 validationID);
error DelegatorIneligibleForRewards(bytes32 delegationID);
```

Recommendations:

Remove unused errors

Kite AI: Resolved with `@e224fbeb89 ...`

Zenith: Verified.